YOURNAME – SUNETID
CS143 Spring 2026 – Written Assignment 4

This assignment covers code generation, operational semantics, and optimization. You may discuss this assignment with other students and work on the problems together. However, your write-up should be your own individual work, and you should indicate in your submission who you worked with, if applicable. Assignments can be submitted electronically through Gradescope as a PDF by Tuesday, June 2, 2026 at 11:59 PM PDT. A LaTeX template for writing your solutions is available on the course website.

1. Programming in Assembly

   Consider the following fictional hardware architecture consisting of two signed 64-bit integer registers `x`, `y`, and a large stack where each stack element holds a signed 64-bit integer.

   The hardware architecture supports the following instructions:

   - `push r`: Given register `r`, push the value of register `r` to the top of the stack.
   - `pop r`: Given register `r`, pop the top of the stack into register `r`.
   - `swap`: Swap the value at the top of the stack with the value right beneath it.
   - `add r1 r2`: Given two registers `r1`, `r2`, compute $r1 + r2$ and store it into register `r1`.
   - `sub r1 r2`: Given two registers `r1`, `r2`, compute $r1 - r2$ and store it into register `r1`.
   - `mul r1 r2`: Given two registers `r1`, `r2`, compute $r1 \times r2$ and store it into register `r1`.
   - `div r1 r2`: Given two registers `r1`, `r2`, compute $r1$ / $r2$ and store it into register `r1`.
   - `jz c`: Given signed 64-bit integer constant literal `c`, if the top of the stack is zero, jump to `c` instructions *after* this instruction. Otherwise, execute the next instruction. For example, `jz 0` is a no-op instruction since the next instruction will be executed no matter the value of the stack top (unless the stack is empty, in which case the machine crashes). As another example, if the stack top is zero, `jz -1` would cause an infinite loop when executed, since it jumps to itself.
   - `print r`: Given register `r`, print its value to console.
   - `halt`: Stop execution.

   All arithmetic operators have the same behavior as C `int64_t` arithmetic, though you should normally not need to worry about this for this problem.

   Your program starts with an empty stack, with register `x` holding an integer value $1 \leq n \leq 19$, and register `y` holding zero.

   You should write a program that, when executed, prints out n lines and halt, where the i-th (1-indexed) line should contain the value of the factorial of i (for example, $4! = 24$).

   Your program should contain one instruction per line. You can also write comments: everything after a `#` character in the same line will be treated as comments and discarded. In programming such an exotic architecture, comments will be very helpful for you to keep track of your program's design and invariants: make use of them!

   You can find a simulator of the architecture at Stanford Myth location

```
/afs/ir/class/cs143/bin/wa4_2026/cs143wa4p1
```

On Stanford Myth, run

```
/afs/ir/class/cs143/bin/wa4_2026/cs143wa4p1 <prog_file> <n>
```

to run your program with register `x` holding value n. Alternatively, you can run

```
/afs/ir/class/cs143/bin/wa4_2026/cs143wa4p1 <prog_file> <n> --trace
```

to additionally print out the machine state after executing each instruction, which will be helpful for you to debug your program.

When you are confident with your program, run

```
/afs/ir/class/cs143/bin/wa4_2026/cs143wa4p1 <prog_file> test
```

to run the staff's tests. If all tests passed, it will print out a secret string. **Be sure to include this secret string in your solution!**

Solution:

The secret string is:

Your program:

2. Single Static Assignment (SSA) Form

Single static assignment (SSA) is an intermediate representation (IR) form used by virtually every production compiler today (LLVM, GCC, V8, HotSpot, and so on). Compared to the IR you saw in lecture, an SSA IR imposes two additional rules:

(a) Each variable is assigned *exactly* once in the entire IR.
(b) Every use of a variable must be preceded (statically) by its assignment.

For instance, the IR basic block

$$a := 5$$
$$a := a - 1$$
$$a := a \times a$$

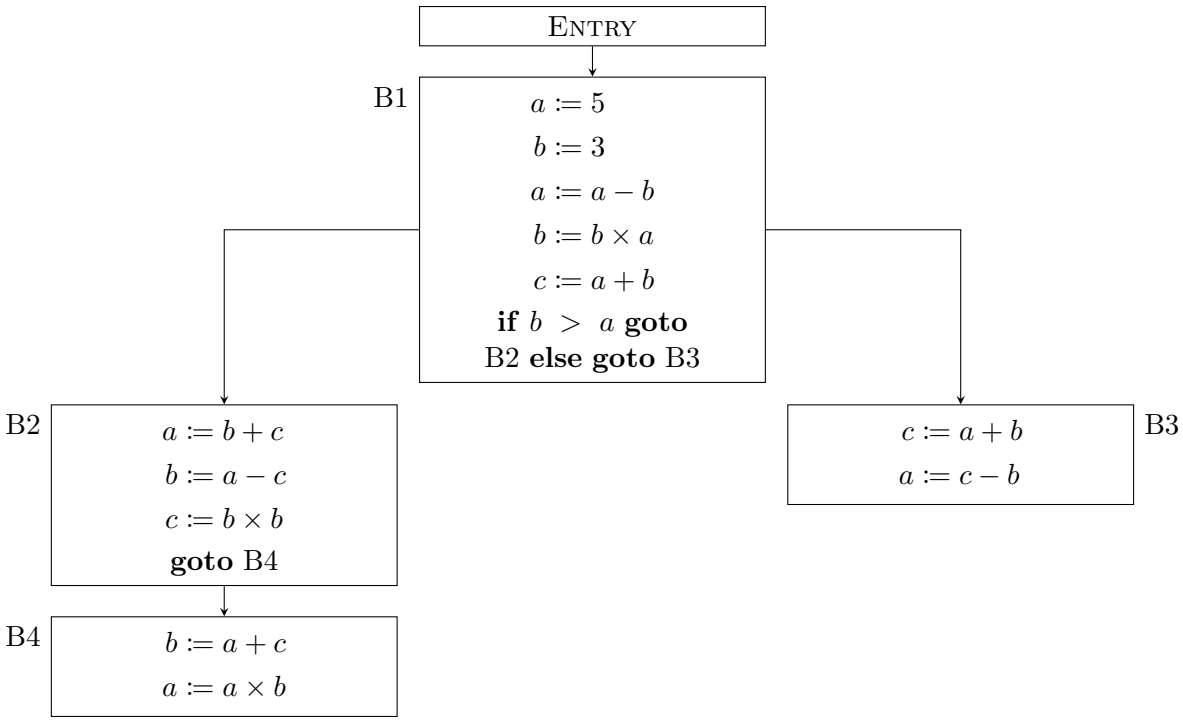
violates rule 1 because a is assigned three times.

The first key idea for converting an IR to SSA form is straightforward: every time a variable is assigned, give it a new "version" (a fresh subscript), and rewrite every subsequent use to refer to the most recently created version. The block above becomes:

$$a_1 := 5$$
$$a_2 := a_1 - 1$$
$$a_3 := a_2 \times a_2$$

When the control flow graph (CFG) has the shape of a tree (no joins), this renaming strategy by itself is sufficient.

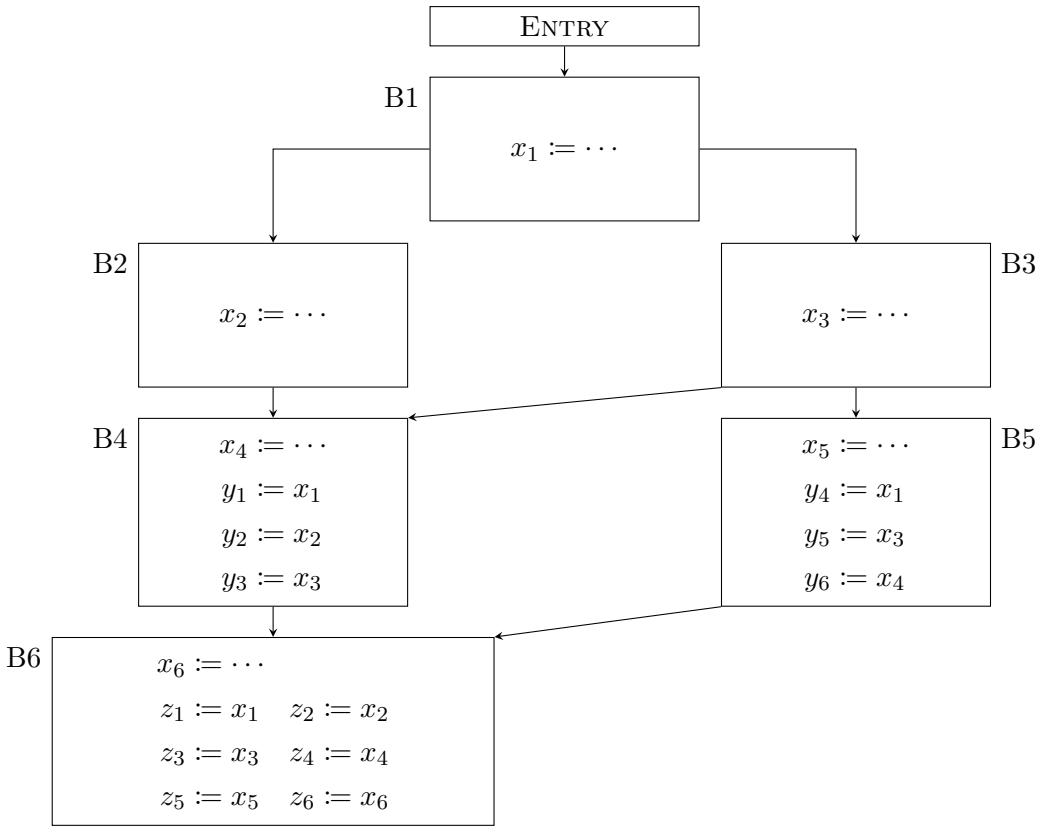
(Task 2.A) As a warm-up, convert the following IR to SSA form:



Now that each variable in an SSA IR is assigned exactly once, this single assignment may naturally be regarded as the *definition* of the variable. The second SSA rule then reads as: a variable must be defined before it is used.

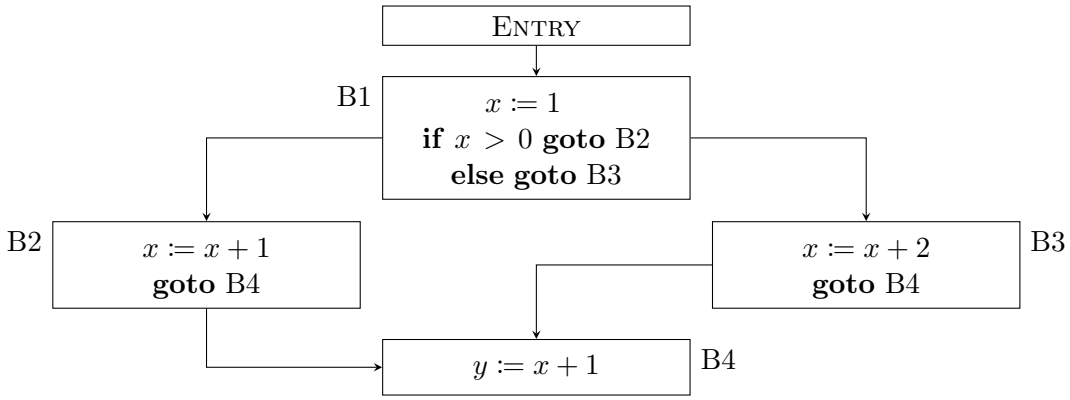
This is a *static* property of the IR: if variable v is defined in basic block X and used in basic block Y, then every path from the entry block to Y in the CFG must go through X (and if $X = Y$, the definition must lexically precede the use in the block). This means we are guaranteed, at compile time, that any use of v is preceded by a valid assignment along every possible execution.

(Task 2.B) Consider the following IR (only the relevant statements are shown; the rest of each block has been omitted for clarity):

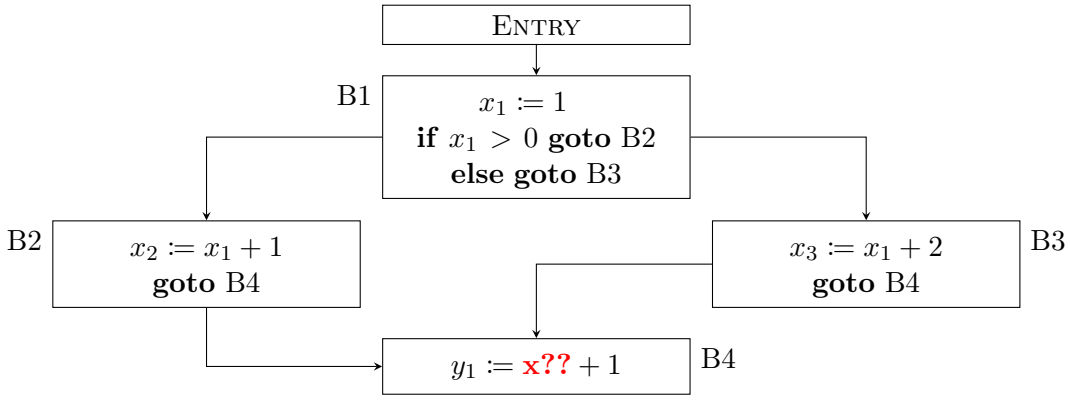


The CFG edges are: B1→B2, B1→B3, B2→B4, B3→B4, B3→B5, B4→B6, B5→B6. Assume the definitions of $x_1, \ldots, x_6$ are all valid. Among $y_1, \ldots, y_6$ and $z_1, \ldots, z_6$, which variables are ill-defined in an SSA IR?

Simply renaming each assignment to a fresh variable is not always enough to produce a valid SSA IR. As an example, consider the IR below:



Applying the renaming process by itself produces:



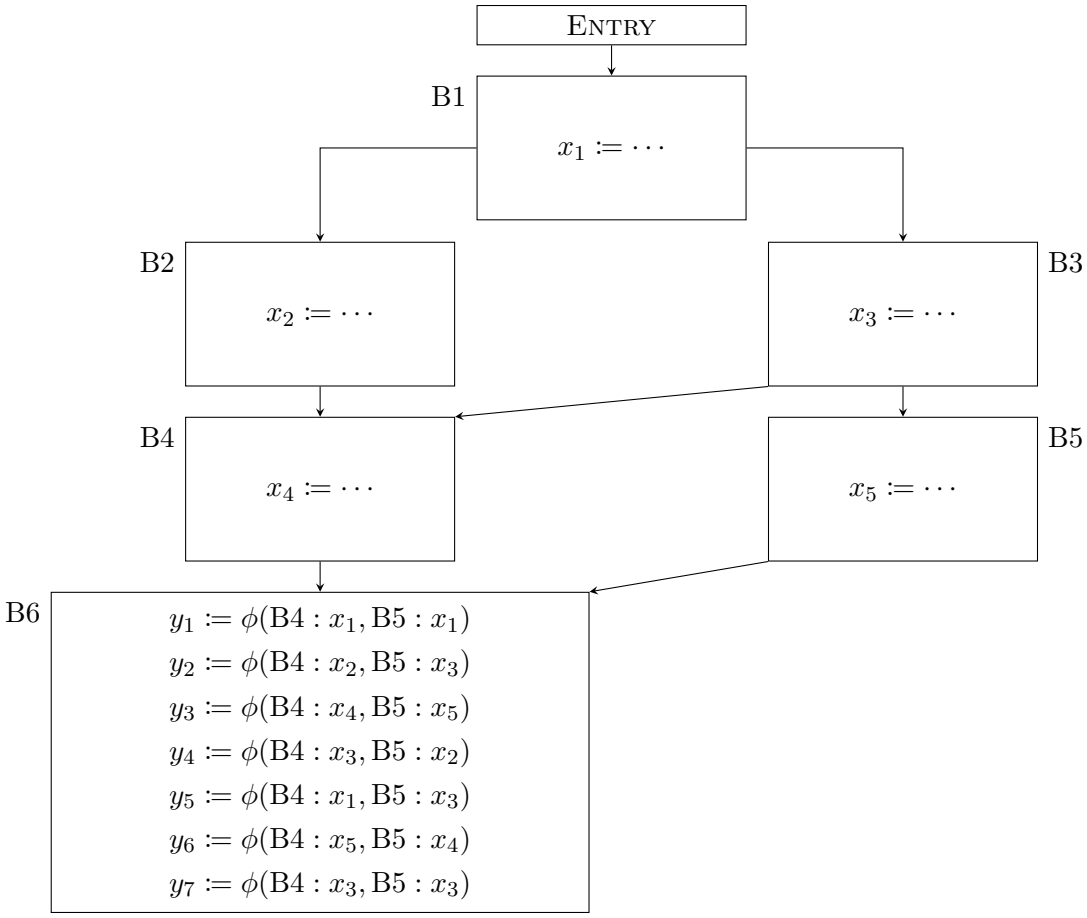
In block B4, the value of x depends on which predecessor we came from, so neither x_2 nor x_3 alone is correct.

To handle this situation, SSA introduces *ϕ-nodes*. If block B has n direct predecessors $P_1, \ldots, P_n$, then a ϕ-node in B takes n arguments $v_1, \ldots, v_n$; when control reaches B from P_i, the ϕ-node evaluates to v_i.

We write a ϕ-node as $\phi(P_1 : v_1, \ldots, P_n : v_n)$. As a convention, a ϕ-node must always appear as the entire right-hand side of an assignment — it cannot be combined with other operators in the same expression. For example, $x \coloneqq \phi(P_1 : v_1, P_2 : v_2) + 1$ is not allowed, but you can split it as $x_1 \coloneqq \phi(P_1 : v_1, P_2 : v_2)$; $x_2 \coloneqq x_1 + 1$. Furthermore, every ϕ-node assignment in a block must appear before any non-ϕ assignment in that block.

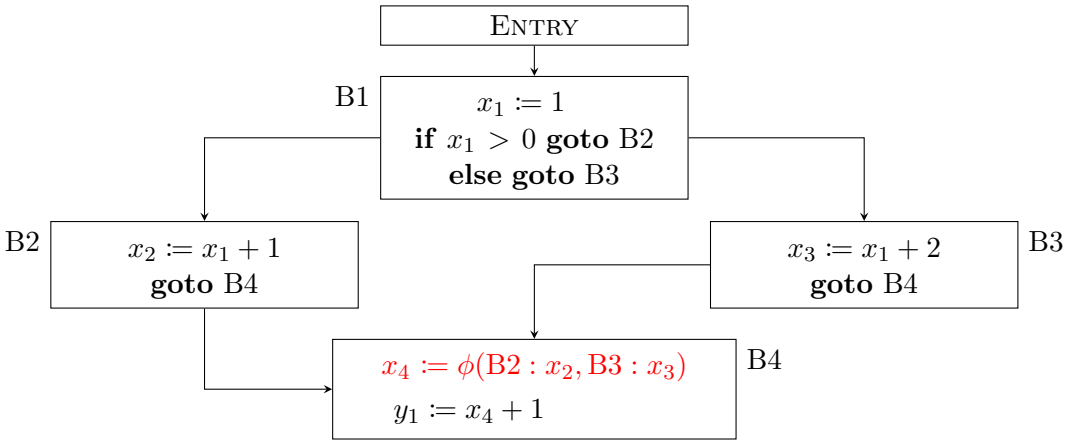
ϕ-nodes are also subject to a def-before-use rule, but in a slightly more refined way. For each $1 \leq i \leq n$, if v_i is defined in basic block X, then every path from the entry block to P_i must go through X. This way, whenever control flows from P_i into B, the v_i that gets selected by the ϕ-node is guaranteed to have a defined value.

(Task 2.C) Consider the following IR (only relevant parts shown):



The CFG edges are: B1→B2, B1→B3, B2→B4, B3→B4, B3→B5, B4→B6, B5→B6. Assume the definitions of $x_1, \ldots, x_5$ are all valid. Which of $y_1, \ldots, y_7$ are ill-defined?

With ϕ-nodes we can now convert the earlier joining example to SSA form:



Can we automate this conversion? If we are willing to be wasteful with the number of ϕ-nodes we insert, there is a fairly simple procedure:

(a) Add a new basic block at the very start of the program; inside it, initialize every variable to `UNDEFINED`. Make this new block the entry block, with a single edge to what used to be the entry block.

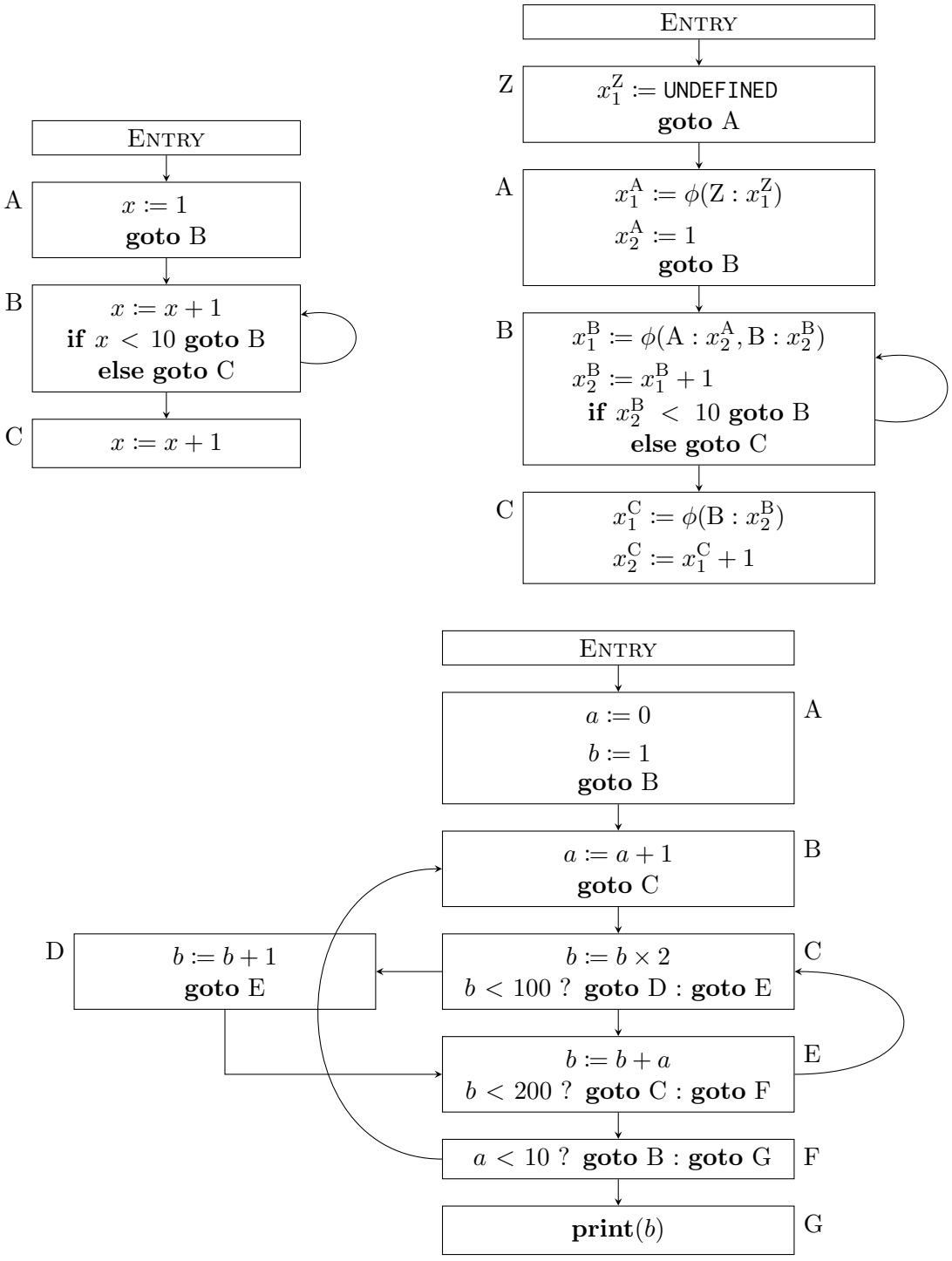
(b) For every basic block B other than the new entry block, and for every variable x, insert an assignment $x_1^{\text{B}} \coloneqq \phi(P_1 : \square, \ldots, P_n : \square)$ at the very top of B (where $P_1, \ldots, P_n$ are the direct predecessors of B). The $\square$ slots will be filled in by step 4.

(c) Within each basic block, do the simple renaming from Task 2.A: each non-ϕ assignment gets a fresh left-hand-side name, and every variable on the right-hand side is rewritten to its most recent version (which always exists, thanks to the start-of-block ϕ-nodes we just inserted).

(d) Go back to each ϕ-node $x_1^{\text{B}} \coloneqq \phi(P_1 : \square, \ldots, P_n : \square)$ from step 2 and replace each $\square$ at slot P_i with the most recent version of x at the end of block P_i.

As a worked example, the algorithm transforms the non-SSA IR on the left below into the SSA IR on the right:

(Task 2.D) Apply the algorithm to the following IR to produce an SSA IR. Make sure you follow the algorithm exactly as described.



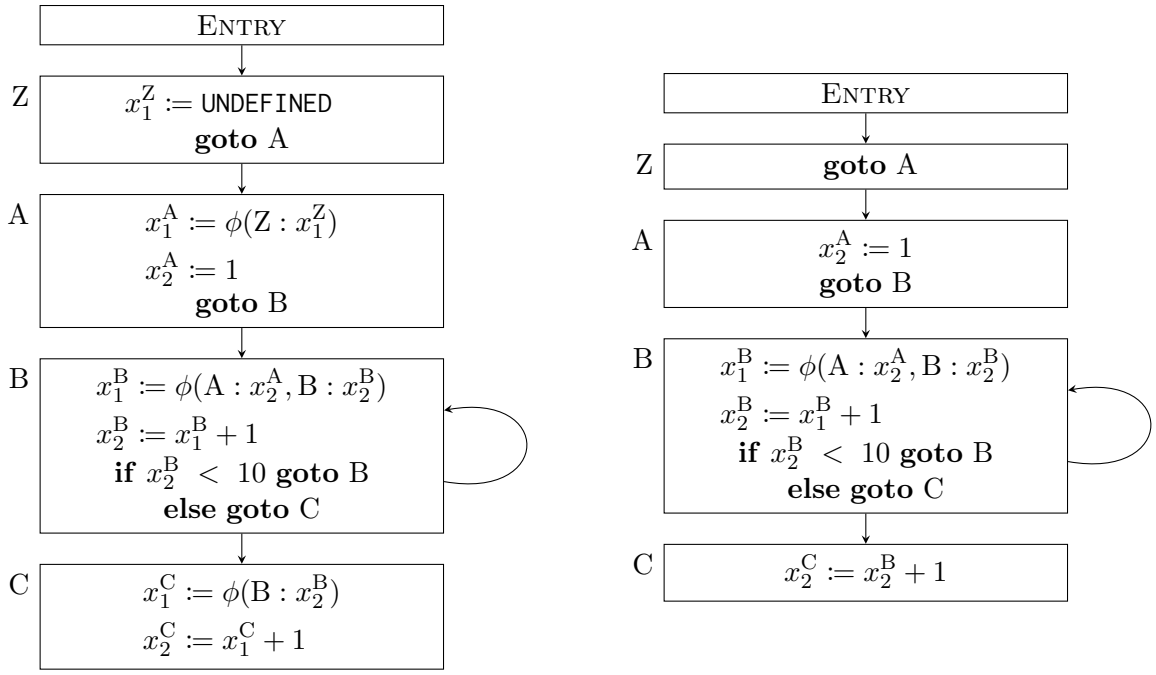
The CFG edges are: Entry→A, A→B, B→C, C→D, C→E, D→E, E→C, E→F, F→B, F→G.

(Task 2.E) Explain why the algorithm always produces a valid SSA IR (not just for the example in Task 2.D, but in general). Specifically, argue that: every variable is defined exactly once, every variable satisfies the def-before-use property, and every ϕ-node satisfies the def-before-use property.

(Task 2.F) Dead code elimination is straightforward on an SSA IR. Because SSA guarantees

that each variable is assigned exactly once, and all uses must occur after the assignment, an assignment is dead code (and can safely be removed) precisely when its right-hand side has no side effect and its left-hand-side variable has no use anywhere in the IR. We can apply this rule repeatedly until no more assignments can be removed. Perform dead code elimination on the SSA IR you produced in Task 2.D. Which variables are dead? (You do not need to show the IR after elimination.)

Not every ϕ-node inserted by the algorithm is actually necessary. For example, in the simple-loop SSA IR we produced earlier (reproduced for convenience on the left below), every use of x_1^{C}, which is defined as $x_1^{\mathrm{C}} \coloneqq \phi(\mathrm{B} : x_2^{\mathrm{B}})$, can be replaced with x_2^{B} without changing the program's behavior. Once we do so, x_1^{C} has no remaining uses; since ϕ-nodes have no side effects, dead code elimination removes the ϕ-node assignment entirely. The right figure shows the simplified SSA IR after replacing uses of x_1^{C} with x_2^{B} and removing the dead variables x_1^{Z}, x_1^{A}, and x_1^{C}.



(Task 2.G) For the SSA IR you obtained in Task 2.F, eliminate as many ϕ-nodes as possible using the technique just illustrated, and write down the resulting simplified SSA IR. (Hint: exactly four ϕ-nodes should remain. Equivalently, you can also approach this by directly identifying which four ϕ-nodes you want to *keep*.) For ease of grading, please do not rename any variable that survives the elimination: all surviving variables should retain the names they had in Task 2.D.

Closing thoughts: this problem set has walked you through the definition of SSA and a complete, correct algorithm for converting a non-SSA IR into SSA form. The algorithm you executed in Task 2.D is, however, far from optimal — both in running time and in the number of ϕ-nodes inserted. In real compilers, the algorithm of choice is Cytron et al.'s fast SSA construction algorithm (introduced in 1989), which is dramatically faster and inserts far fewer ϕ-nodes on real-world programs than our naive scheme[1]: when the input program uses only structured control flow (no `goto`/`break`/`continue`), Cytron's algorithm even runs in linear time in the size of the program (and as a corollary inserts at most $O(n)$ ϕ-nodes). The path to that better algorithm starts with formalizing the observations you just made in Task 2.G — namely, when is a ϕ-node actually needed? — and using appropriate data structures (most notably, the *dominance frontier*) to detect those situations efficiently. The full development is beyond the scope of this problem set; if you are interested, the Wikipedia page on SSA is a good starting point.

[1] One can construct adversarial inputs on which Cytron's algorithm reaches the same theoretical worst-case complexity as our naive algorithm. In practice, however, what matters for a compiler is performance on real-world programs, not artificial pathological cases.

3. Consider the following assembly-like pseudo-code, using 6 temporaries (abstract registers) a to f:

```
1       a := b + d
2       b := a + d
3       d := a - b
4       c := d + d
5       if c > 100:
6       c := c + d
7       else:
8       d := 1
9       e := d - c
10      f := e - c
```

(a) At each program point, list the variables that are live. Note that **b** and **d** are inputs for the given code and **f** is a live value on exit.

Solution:

(b) Draw the register interference graph between temporaries in the above program as described in class.

Solution:

(c) Provide a lower bound on the number of registers required by the program induced from the interference graph. Can you explain why?

Solution:

(d) Using the algorithm described in class, provide a coloring of the graph in part(b). The number of colors used should be your lower bound in part (c). Provide the final k-colored graph (you may use the tikz package to typeset it or simply embed an image), along with the order in which the algorithm colors the nodes.

Solution:

(e) Based on your coloring, write down a mapping from temporaries to registers (labeled r1, r2, etc.).

Solution: